

FLUIDWORKS: REAL-TIME CFD VISUALIZATION PLATFORM

*The Mini Project report submitted in partial fulfillment of the
requirements for the award of the degree of*

Bachelor of Technology

in

Artificial Intelligence & Machine Learning

by

Abhinav Manoj (Reg. No. MCE23AIM003)

Niranjana J (Reg. No. MCE23AIM047)

Abhiram Vijaykumar Kaimal (Reg. No. MCE23AIM004)

Rohan Tom Reji (Reg. No. MCE23AIM052)



**Marian Engineering College
Trivandrum
Kerala**

March 2026

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Marian Engineering College
Kazhakuttom, Trivandrum - 695582



CERTIFICATE

This is to certify that the report entitled “**FluidWorks: Real-Time CFD Visualization Platform**” is a bonafide record of the Mini Project presented by Abhinav Manoj (Reg. No. MCE23AIM003), Niranjan J (Reg. No. MCE23AIM047), Abhiram Vijaykumar Kaimal (Reg. No. MCE23AIM004) and Rohan Tom Reji (Reg. No. MCE23AIM052) under our supervision and guidance. The report has been submitted to the Department of Artificial Intelligence & Machine Learning of Marian Engineering College, Kazhakuttom, Trivandrum in partial fulfillment of the award of the Degree of Bachelor of Technology in Artificial Intelligence & Machine Learning, during the year 2025-2026.

Dr. Aswathy S U
Supervising Guide
Assistant Professor
Artificial Intelligence &
Machine Learning
Marian Engineering College
Trivandrum

Prof. Balu John
Head of the Department
Artificial Intelligence &
Machine Learning
Marian Engineering College
Trivandrum

ACKNOWLEDGEMENT

We express our sincere gratitude to our Principal **Dr. Abdul Nizar M.**, to **Prof. Balu John**, Head of the Department of Artificial Intelligence & Machine Learning, and to the faculty members who supported the completion of this work. We record our special thanks to our supervising guide **Dr. Aswathy S. U** for the steady review, practical suggestions, and academic guidance that shaped both the software and this report. We also acknowledge our classmates and friends for their feedback during testing and demonstration. The project demanded work across simulation logic, user-interface design, documentation, and validation, and their support made that process manageable.

ABSTRACT

FluidWorks: Real-Time CFD Visualization Platform is a Unity-based desktop platform for interactive CFD-style visualization. The current implementation centers on a maintained external-aero wind-tunnel path supported by a GPU grid solver, streamline and slice visualizations, vehicle-aware engineering estimates, runtime 3D model loading, and export-oriented reporting tools. The repository also includes an internal-flow workflow with explicit boundary assignment, surface pressure and friction views, rotating-machinery support, and free-surface experimentation. In addition to the simulation stack, the project contains a lightweight AI/ML-assisted geometry assessment layer that predicts aerodynamic tendencies from extracted shape features and presents actionable recommendations in the analysis interface. This report reorganizes the earlier FluidWorks documentation into the exact mini-project format required by the new reference template while preserving the established figures, tables, technical framing, and source-grounded explanations. The document covers background, system design, solver workflow, AI/ML components, testing strategy, observed outcomes, limitations, and representative appendix listings from the current repository.

Contents

List of Figures	iii
List of Tables	iv
1 Introduction	1
1.1 Overview	1
1.2 Motivation	1
1.3 Problem Definition	2
1.4 Objectives	2
1.5 Scope of the Project	2
1.6 Chapter Details	3
2 Literature Survey and Technical Background	4
2.1 Interactive CFD as an Engineering Need	4
2.2 GPU-Oriented Simulation in a Teaching Context	4
2.3 Visualization as a Core Engineering Layer	4
2.4 Geometry-Aware Evaluation	5
2.5 Educational Relevance	5
2.6 Difference from Conventional CFD Packages	5
3 Proposed System	7
3.1 High-Level Architecture	7
3.2 Implemented Workflow Summary	8
4 Simulation Workflows and Implementation	9
4.1 External Aero: Current Wind-Tunnel Stack	9
4.2 Wind-Tunnel Visualization Contract	9
4.3 Streamline Rendering Quality	10
4.4 Vehicle-Aware Engineering Outputs	10
4.5 Internal Flow: Updated Pipeline	10
4.6 Boundary Assignment and Surface Views	11
4.7 User Workflow	11
4.8 Implementation Observations	11
5 AI/ML Components Included in the Project	13
5.1 Why AI/ML Was Included	13
5.2 Aero Geometry Analysis Pipeline	13
5.3 ML Predictor Design	13
5.4 Feature Importance and Explainability	14

5.5	Improvement Suggestions	14
5.6	Analysis Interface Integration	14
5.7	Auto Segmentation Workflow	15
5.8	AI/ML Workflow Summary	15
5.9	Value and Limitations of the AI/ML Layer	16
6	Testing Strategies & Result Achieved	17
6.1	Validation Approach	17
6.2	Compile and Build Discipline	17
6.3	Representative Validation Targets	18
6.4	Results Interpretation	18
6.5	Observed Strengths	18
6.6	Limitations	19
6.7	Why the Revised Report Matters	19
7	Applications, Discussion and Future Enhancements	20
7.1	Practical Applications	20
7.2	Discussion	20
7.3	Future Enhancements	20
8	Conclusion & Future Work	22
	Bibliography	23
	Appendix A Wind-Tunnel Runtime Driver Excerpt	24
	Appendix B Navier-Stokes Grid Solver Excerpt	28
	Appendix C Compute Shader Excerpt	32
	Appendix D AI/ML Predictor Excerpt	36
	Appendix E Geometry Analysis Excerpt	40
	Appendix F Project Save Manager Excerpt	44
	Appendix G Project Data Model Excerpt	46

List of Figures

3.1	High-level architecture of the current FluidWorks runtime	7
4.1	Typical user workflow in FluidWorks	11
5.1	AI/ML-assisted model quality workflow in FluidWorks	15

List of Tables

3.1	Current simulation workflows in FluidWorks	8
4.1	Wind-tunnel visualization expectations reflected in the repository . .	10
6.1	Representative validation targets in the current repository	18

Chapter 1

Introduction

1.1 Overview

FluidWorks is a real-time CFD-style visualization application built in Unity for fast engineering interpretation. The project does not target certification-grade numerical fidelity. Its purpose is different and more practical for a mini-project setting: load geometry quickly, configure a scenario with minimal friction, obtain readable flow visuals, and support the result with diagnostics that a reviewer can understand without stepping through a full industrial CFD workflow.

The current repository is not a thin prototype. It contains an actively maintained external-aerodynamics path, a dedicated internal-flow path, scene-level workflow selection, solver-side diagnostics, multiple visualization modes, export support, and a set of UI panels that expose the simulation in a structured way. The older product name AeroFlow still appears in namespaces and some file identifiers, but the present project identity is **FluidWorks**, and this report reflects the current state of that codebase.

1.2 Motivation

Traditional CFD packages are powerful, but they are often heavy in three ways: setup time, computational cost, and interpretation effort. Even before the first result appears, the user may have to prepare the mesh, assign boundaries carefully, choose solver models, monitor convergence, and post-process the output. That workflow is appropriate for detailed engineering studies, but it is not ideal for quick classroom demonstrations, early-stage concept reviews, or interactive exploration of imported geometry.

FluidWorks was motivated by that gap. The goal was to build a system that remains technically grounded while still feeling immediate. A user should be able to import a model, select a relevant simulation template, inspect the resulting field visually, and relate those visuals to values such as drag force, pressure drop, wall shear, divergence, or predicted aerodynamic quality. In short, the software should shorten the distance between “load model” and “understand what is happening.”

1.3 Problem Definition

The central engineering problem addressed in this project can be expressed as follows:

How can a desktop application provide visually meaningful and technically defensible fluid-flow analysis across multiple engineering scenarios while remaining responsive enough for interactive use on commodity hardware?

This problem is not solved by simply showing particles or by simply implementing equations. The project must connect geometry import, solver execution, boundary handling, user controls, visualization, diagnostics, and output generation. If any one of those layers is weak, the application stops being useful as a complete engineering tool.

1.4 Objectives

The present implementation is aimed at the following objectives:

1. build a single desktop interface that can host multiple flow-analysis workflows;
2. support runtime import of engineering geometry so that the same application can evaluate different models without recompilation;
3. provide a GPU-accelerated external-aero wind-tunnel path that stays visually stable during interaction;
4. provide an internal-flow path with explicit opening detection and boundary assignment;
5. expose diagnostics that support the visuals instead of leaving them as decorative output;
6. include an AI/ML-assisted geometry assessment layer for rapid model-quality feedback;
7. support capture and export of results in a report-friendly form; and
8. maintain a modular implementation that can be extended in future iterations.

1.5 Scope of the Project

The project scope covers interactive external aero, internal flow, rotating-machinery visualization, and free-surface experimentation. Among these, the wind-tunnel path is the most mature and acts as the baseline solver workflow in the repository. The internal-flow path extends the software beyond vehicle-style demonstrations and introduces boundary-assignment logic, surface pressure view, and friction-oriented interpretation. The rotating-machinery and free-surface paths broaden the template catalog and show that the architecture can support more than one simulation family.

The scope does not include industrial turbulence-model depth, mesh-quality optimization pipelines, or certification-quality validation against standard benchmark

datasets. Those areas would require much more numerical and hardware investment than is reasonable for this project. The emphasis here is educational usefulness, interactive response, and technically sensible behavior.

1.6 Chapter Details

This report is organized to match the mini-project reference format while retaining the engineering depth of the earlier FluidWorks report. Chapter 2 summarizes the technical background that motivated the project design. Chapter 3 explains the proposed system architecture and workflow. Chapter 4 discusses the tools and technologies used in the implementation. Chapter 5 presents the active simulation stacks in the repository. Chapter 6 describes the AI/ML components that are actually included in the project. Chapter 7 records testing strategy and observed results. Chapter 8 discusses applications, limitations, and future scope. Chapter 9 concludes the report, and the appendices provide representative listings from the current codebase.

Chapter 2

Literature Survey and Technical Background

2.1 Interactive CFD as an Engineering Need

Computational Fluid Dynamics is usually introduced through the Navier-Stokes equations, discretization methods, and boundary-value problems [1, 2, 4]. In practice, however, users encounter CFD through tools and workflows. For students and early-stage designers, the challenge is rarely the existence of equations alone. The challenge is the length of the route from geometry to insight. An interactive CFD-style platform shortens that route by reducing setup friction and emphasizing readable outputs.

This idea is increasingly relevant in education. Students understand pressure recovery, wake formation, blockage, and separation more quickly when they can change geometry or inlet settings and immediately see the result. The same is true for internal flow: pressure drop and wall-shear ideas become more concrete when the visualization and diagnostics move together.

2.2 GPU-Oriented Simulation in a Teaching Context

GPU compute is a natural choice for a project that values responsiveness. Structured grids, buffer-based updates, and repeated kernel execution map well to compute shaders [7, 6]. Even when the solver is simplified relative to a high-end CFD suite, the ability to update a field repeatedly on the GPU provides enough speed for interactive exploration.

In a mini-project context this matters for two reasons. First, it keeps the project technically ambitious by moving the heavy iterative work away from the CPU. Second, it lets visualization remain coupled to the solver instead of turning into an offline post-processing step. That coupling is a major reason why FluidWorks feels like a tool rather than a static demonstration.

2.3 Visualization as a Core Engineering Layer

Many student projects treat visualization as an afterthought. FluidWorks treats it as a primary engineering layer. Streamlines, slices, particles, and surface contours

are not merely aesthetic additions; they are the main mechanism by which the user interprets the field. A clean diagnostic number can tell the user that drag has changed, but the visualization tells the user *why* the change likely happened. That is especially important when the software is used for review, explanation, and comparison.

2.4 Geometry-Aware Evaluation

Another important background idea is that not every useful assessment needs a full flow solve. Some engineering judgments can be approximated from geometry itself. If a model is very blocky, highly asymmetric, rough, or poorly tapered, the software can flag that early. FluidWorks uses this idea in its AI/ML-assisted model-quality analysis. The system extracts interpretable geometric features and feeds them to a small neural predictor that estimates trends such as drag tendency, separation risk, downforce potential, and overall efficiency score [9, 10, 11].

2.5 Educational Relevance

The project is academically useful because it sits at the intersection of several areas commonly taught separately:

1. fluid mechanics, through drag, lift, pressure, vorticity, wall shear, and pressure drop;
2. numerical methods, through discretized solver updates and divergence control;
3. computer graphics, through scene-space rendering and compute-shader execution;
4. software engineering, through modular runtime architecture and export workflows; and
5. artificial intelligence, through lightweight predictive assessment and geometry-driven recommendations.

2.6 Difference from Conventional CFD Packages

FluidWorks does not attempt to replace commercial or research CFD packages. It intentionally trades some numerical sophistication for interactivity, usability, and readability. The design assumptions are straightforward:

1. the user should obtain meaningful output quickly;
2. controls and diagnostics should remain in the same workspace;
3. the software should remain responsive on typical desktop hardware; and
4. the output should be suitable for explanation, demonstration, and report generation.

Those assumptions define the project clearly and keep expectations realistic. The value of FluidWorks lies in the balance it strikes between technical grounding and usability.

Chapter 3

Proposed System

3.1 High-Level Architecture

FluidWorks is organized as a layered runtime system. The user begins in the UI, chooses or loads a project, selects a workflow template, configures the scenario, and then interacts with a simulation driver that orchestrates the solver, diagnostics, and active visualizers.

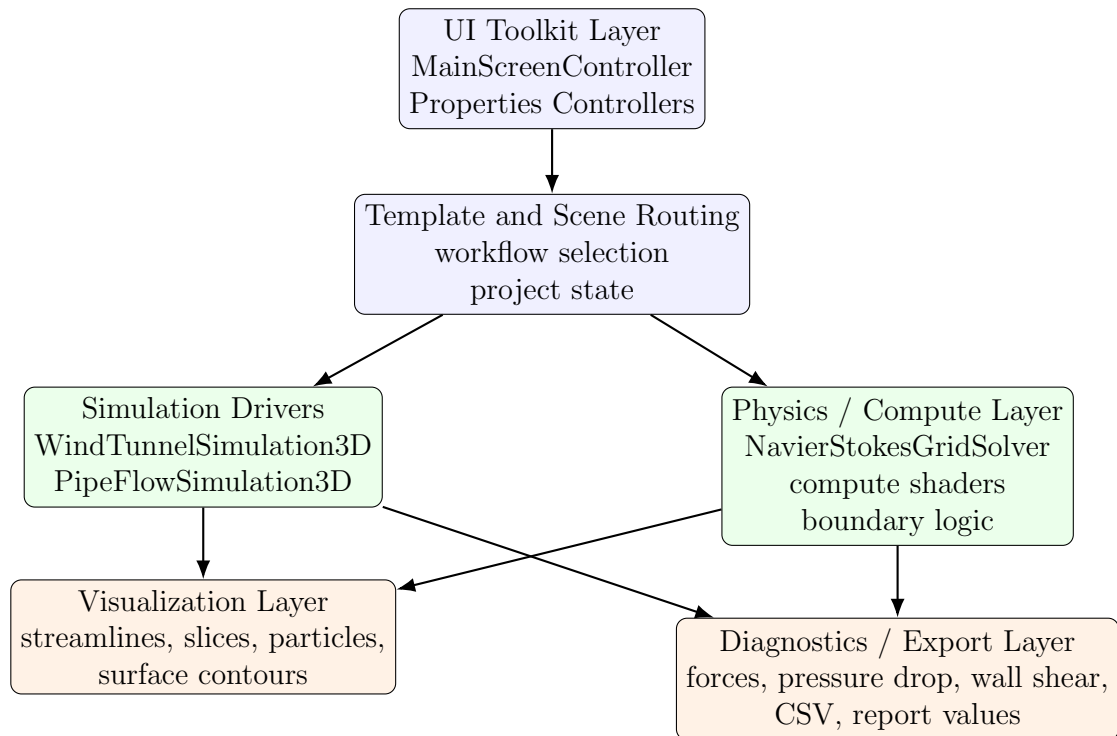


Figure 3.1: High-level architecture of the current FluidWorks runtime

3.2 Implemented Workflow Summary

Table 3.1: Current simulation workflows in FluidWorks

Workflow	Primary Purpose	Current Notes
External Aero	Wind-tunnel style external flow around a vehicle or imported body	Most mature path; GPU Navier-Stokes grid solver, vehicle-aware outputs, multiple visualization contracts
Internal Flow	Pipe and duct style flow analysis	Mesh-driven boundary handling, opening classification, streamlines, and surface pressure / friction visualization
Rotating Machinery	Rotor, fan, propeller, and turbine style visualization	Runtime geometry workflow with rotating-region controls and machine-oriented diagnostics
Free Surface	Dam-break or splash-style study	GPU particle-style free-surface path for educational and demonstrative use

Chapter 4

Simulation Workflows and Implementation

4.1 External Aero: Current Wind-Tunnel Stack

The wind-tunnel path is the strongest implementation in the repository and acts as the baseline engineering workflow. `WindTunnelSimulation3D.cs` is the runtime driver. It owns initialization, playback, state validation, and solver orchestration. The repository guidance also makes it clear that the tunnel flow axis must remain explicit through `flowAxis`. The older longest-axis heuristic is not to be reintroduced because it produced ambiguous behavior in nonstandard tunnel shapes.

The active solver path uses `NavierStokesGridSolver.cs` together with the compute shader `NavierStokes3D.compute`. This pair handles field stepping, obstacle coupling, pressure-related correction, and diagnostics such as divergence and vorticity. Obstacle interaction depends on `ObstacleVoxelizer.cs` as well, which converts loaded geometry into a representation the solver can use.

4.2 Wind-Tunnel Visualization Contract

The visualization contract in the repository is explicit and worth documenting because it prevents mode confusion:

1. **Effects** is the stylized wake mode and is intended to give a readable impression of flow motion around a loaded vehicle.
2. **Velocity** uses compact solver-driven slices and focused streamlines.
3. **Pressure** combines model pressure visualization with a body-adjacent slice rather than a full-screen slab.
4. **Surface Pressure** remains a body-only contour mode and should not replace the broader pressure view.

This contract matters because visualization quality can easily degrade when a project accumulates overlapping modes that appear similar. FluidWorks avoids that by giving each mode a clear purpose.

Table 4.1: Wind-tunnel visualization expectations reflected in the repository

Mode	Primary Output	Expected Behavior
Effects	Stylized wake and motion impression	Readable body wake, not a raw diagnostic slab
Velocity	Streamlines and compact slices	Solver-driven views with focused interpretation
Pressure	Model pressure plus body-adjacent slice	Local pressure understanding without screen-filling clutter
Surface Pressure	Body-only contour	Additive contour mode separated from the broader pressure view

4.3 Streamline Rendering Quality

The repository notes place unusual emphasis on streamline quality, and that is justified. A weak streamline presentation makes the solver look less credible even when the field is numerically stable. The current expectations are clear:

1. solver data remains the primary velocity source;
2. seed positions use a stable random sequence to avoid frame flicker;
3. lines must be long enough to traverse a meaningful portion of the tunnel;
4. wake regions must not be eliminated by an aggressive speed-kill threshold; and
5. the palette should remain vivid enough to communicate velocity changes.

These details may sound cosmetic, but they are essential to the user experience. A user forms trust in the system largely through stable and interpretable output.

4.4 Vehicle-Aware Engineering Outputs

The wind-tunnel path also exposes derived estimates through the analysis interface. These include drag force, vertical aero force, downforce, center of pressure, pitch, yaw, and roll moments, axle loads, and estimated top speed. The repository guidance explicitly separates those values from solver diagnostics such as divergence error and mean vorticity. That separation is important because the first group is vehicle-oriented interpretation, whereas the second group is field-quality and solver-health feedback.

4.5 Internal Flow: Updated Pipeline

The internal-flow path extends the platform beyond vehicle-style studies. `PipeFlowSimulation3D.cs` manages the domain, solver stepping, and metric sampling for pipe and duct analysis. A significant recent improvement is the explicit role of `BoundaryConditionManager.cs`. Internal flow is much more sensitive to opening

meaning than a simple wind tunnel, so the project now treats boundary assignment as a workflow requirement rather than an invisible assumption.

4.6 Boundary Assignment and Surface Views

The internal-flow workflow uses detected openings, user-assigned inlet and outlet semantics, and surface-driven diagnostics such as pressure and friction visualization. This is one of the strongest signs that the project has matured. Rather than presenting only a volume field, the software now gives the user a body-coupled interpretation of what the field is doing at the wall. For report preparation, this is especially valuable because surface views communicate more directly than abstract field slices.

4.7 User Workflow

In practical use, a typical FluidWorks session follows a predictable path: load or recover a model, choose the relevant template, configure settings, run the solver, inspect the active visual mode, and export a result package if needed.

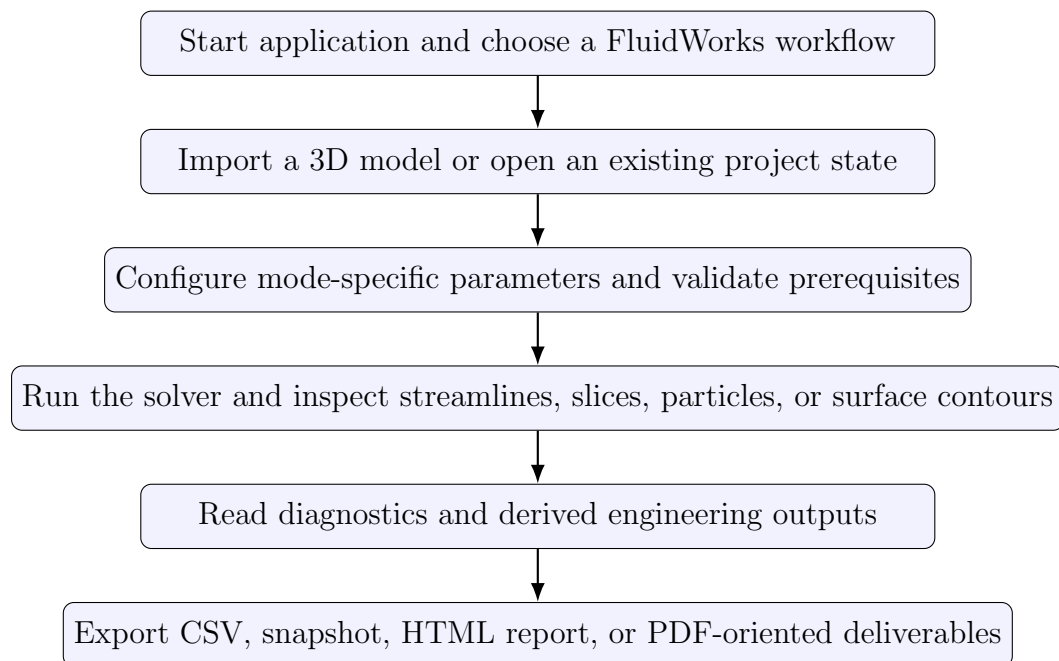


Figure 4.1: Typical user workflow in FluidWorks

4.8 Implementation Observations

Two implementation choices stand out when the repository is read as a complete system. First, the project consistently favors explicit control over hidden heuristics. The explicit flow axis and explicit boundary assignment are both examples of this. Second, the project favors readable engineering output over excessively

dense screens. The visualization contract, the solver diagnostics grouping, and the separation between model-quality assessment and field diagnostics all follow that principle.

Chapter 5

AI/ML Components Included in the Project

5.1 Why AI/ML Was Included

The project belongs to the Artificial Intelligence & Machine Learning domain, so the final report should document where AI/ML is genuinely used. In FluidWorks, AI/ML does not replace the CFD-style solver. Instead, it complements the simulation with a fast geometry-aware assessment layer. This is a sensible design choice because some questions can be answered directly from shape features long before a user studies the full flow field in detail.

5.2 Aero Geometry Analysis Pipeline

The AI/ML functionality is centered on two repository files: `AeroGeometryAnalyzer.cs` and `AeroMLPredictor.cs`. The first file extracts geometric descriptors from the currently loaded model. The second file applies a lightweight neural network to those descriptors and returns a set of predictive values.

The extracted feature set is deliberately interpretable. It includes aspect ratio, slenderness ratio, frontal blockage ratio, volume efficiency, surface roughness, symmetry score, underside clearance, and rear taper ratio. These are not arbitrary neural inputs. They are quantities with direct aerodynamic meaning, which makes the resulting analysis easier to explain during evaluation.

5.3 ML Predictor Design

The predictor itself is implemented as a small multilayer perceptron with fixed, physics-informed weights. According to the source file comments, the network architecture is:

1. 8 input features;
2. 16 neurons in the first hidden layer with ReLU activation;
3. 8 neurons in the second hidden layer with ReLU activation; and
4. 4 output values passed through a sigmoid-scaled interpretation layer.

The output values are:

1. predicted drag-coefficient tendency;
2. separation-risk estimate;
3. downforce-potential estimate; and
4. overall efficiency score.

This is a sensible design for the project because it keeps inference local, dependency-free, and fast. The implementation does not depend on an external runtime such as ONNX or TensorFlow. That keeps deployment simple and avoids turning the ML component into a separate infrastructure problem. The design also matches the educational value of compact feed-forward models discussed in standard machine-learning texts [9, 10].

5.4 Feature Importance and Explainability

One of the strongest AI/ML-related decisions in the repository is the inclusion of feature-importance estimation. The predictor computes central-difference sensitivity by perturbing each input feature and aggregating output variation. This is valuable for two reasons. First, it prevents the ML result from appearing as an unexplained black-box label. Second, it lets the software convert feature sensitivity into readable breakdown text and improvement suggestions. In that sense the project follows the broader direction of interpretable or explainable ML, where a model should assist human judgment rather than replace it blindly [11, 12].

In practical terms, the user is not shown only a score. The software can report that blockage, roughness, taper, or symmetry had a high influence on the assessment. That makes the AI/ML layer much more appropriate for an academic project because the reasoning remains inspectable.

5.5 Improvement Suggestions

The geometry analyzer converts feature values and ML outputs into ranked design suggestions. The logic in the repository checks for high roughness, high blockage, weak rear taper, low aspect ratio, poor symmetry, very low clearance, and low volume efficiency. It also adds direct warnings when the model predicts high separation risk or high drag tendency. This turns the AI/ML output into something actionable instead of merely descriptive.

5.6 Analysis Interface Integration

The AI/ML outputs are not hidden in code. The analysis interface contains a dedicated “Model Quality Analysis (ML)” group. The UI labels display:

1. overall model quality score;

2. grade;
3. predicted drag-coefficient range;
4. separation risk;
5. downforce potential;
6. ML efficiency score;
7. geometric feature breakdown; and
8. improvement suggestions.

This is important because it shows that the ML component is a user-facing product feature, not just a silent experiment inside the repository.

5.7 Auto Segmentation Workflow

The project also includes an “Auto Segment Model” action in the segmentation interface. This does not function as a large standalone ML system, but it is part of the broader intelligent-assistance story in the application. The segmentation controller can trigger automatic splitting and auto-identification of parts when the imported model needs motion-aware classification. In a workflow that supports rotating machinery and moving parts, this reduces manual setup effort and improves usability.

5.8 AI/ML Workflow Summary

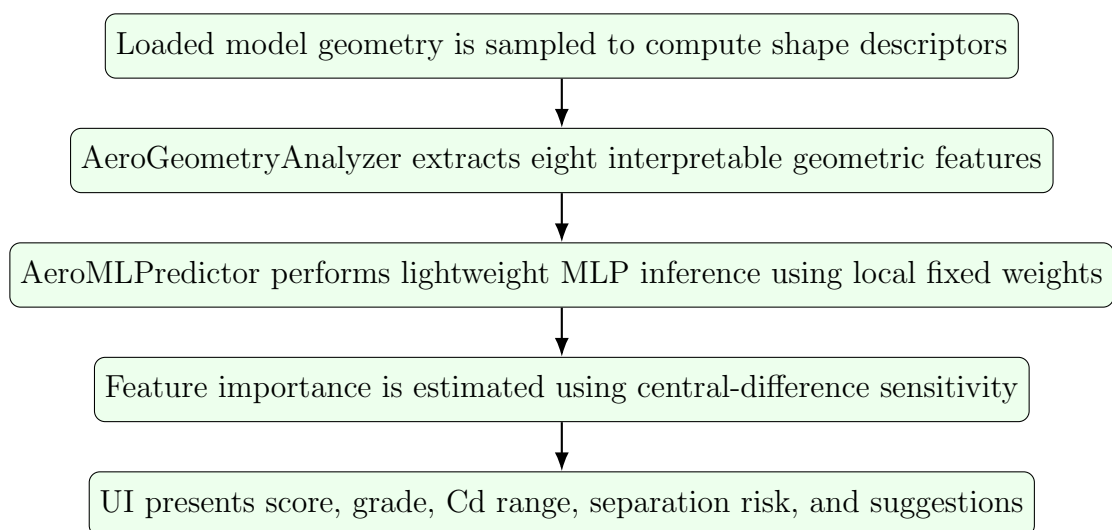


Figure 5.1: AI/ML-assisted model quality workflow in FluidWorks

5.9 Value and Limitations of the AI/ML Layer

The AI/ML component adds clear value because it gives immediate design feedback even before the user spends time interpreting the full simulation result. It also aligns the project more closely with the degree program by introducing a predictive layer that is fast, explainable, and integrated into the main product workflow.

At the same time, its limitations should be stated plainly. The model is lightweight, physics-informed, and hand-embedded into the application. It is not trained online, it does not replace the solver, and it should be understood as an assistive estimator. In the context of this project, that is the correct scope. The strength of the feature is that it improves interaction and interpretation without pretending to be more than it is.

Chapter 6

Testing Strategies & Result Achieved

6.1 Validation Approach

The repository guidance makes it clear that validation should focus on behavior, stability, and technical credibility. For the wind-tunnel path, that means verifying mode switching, readable solver diagnostics, explicit flow-direction handling, reasonable wake behavior, and stable visualization output. For internal flow, it means checking that openings are identified correctly, boundaries are assigned before analysis, and surface pressure or friction views remain consistent with the computed field.

6.2 Compile and Build Discipline

The project notes also require a Unity batch compile check after non-trivial changes and a Windows batch build after compile success. This is an important engineering habit because Unity-based projects can fail at script-compile stage, scene binding stage, or build-export stage even when individual scripts look correct in isolation.

6.3 Representative Validation Targets

Table 6.1: Representative validation targets in the current repository

Workflow	What is checked	Expected outcome
External Aero	Mode switching, yaw response, custom wind direction, turbulence stress	Stable visuals, clear mode replacement, no screen-filling artifacts
External Aero	Solver diagnostics	Readable values for divergence error and mean vorticity
Internal Flow	Boundary assignment and opening detection	Valid inlet and outlet setup before analysis
Internal Flow	Pressure and friction contours	Surface feedback remains tied to computed field behavior
UI / Workflow	Template selection, property binding, export triggers	Controls map correctly to active simulation state
AI / ML	Geometry assessment labels and suggestions	Quality score, grade, feature breakdown, and recommendations update for loaded models

6.4 Results Interpretation

The project outcome is best understood through the kind of insight it gives to the user.

For external aero, the user can inspect stagnation at the front of the model, acceleration along the body, wake formation at the rear, asymmetry under yaw, and the way those patterns relate to drag and load estimates. For internal flow, the user can inspect whether the fluid path remains continuous through a cavity, where pressure loss concentrates, and where wall interaction intensifies. The software does not merely state that something changed; it gives a visual narrative that explains why the change matters.

6.5 Observed Strengths

The strongest achieved results in the current repository are:

1. a mature wind-tunnel path with explicit design rules;
2. stable solver-oriented visual outputs that remain readable in presentation settings;
3. a stronger internal-flow workflow than the earlier report documented;

4. export-friendly metrics and report support; and
5. a meaningful AI/ML-assisted analysis layer instead of a purely symbolic AI mention.

6.6 Limitations

The present build still has clear limitations:

1. it is not a replacement for high-fidelity commercial CFD software;
2. the quality of interpretation is stronger than the claim of absolute numerical precision;
3. high resolutions or unsuitable hardware can reduce performance;
4. some workflows remain less mature than the wind-tunnel baseline; and
5. the AI/ML predictor is intentionally lightweight and should be treated as assistive rather than authoritative.

6.7 Why the Revised Report Matters

The earlier report was adequate as a summary, but it was too short for a full mini-project submission and too light on implementation evidence. This revised version is more appropriate for evaluation because it documents solver organization, workflow structure, visualization rules, AI/ML inclusion, and source-grounded implementation decisions in a format that matches the new reference template.

Chapter 7

Applications, Discussion and Future Enhancements

7.1 Practical Applications

FluidWorks can be applied in several realistic contexts:

1. classroom demonstration of aerodynamic and internal-flow concepts;
2. rapid comparison of alternate body shapes or duct layouts;
3. visual support during mini-project reviews and viva discussions;
4. exploratory analysis before moving to heavier CFD packages; and
5. interactive proof-of-concept work for future engineering software.

7.2 Discussion

One of the strongest aspects of the project is that it joins simulation, interpretation, and presentation into a single workflow. That is often where student engineering tools fall short. A project may solve something numerically but fail to communicate it well, or it may look attractive but fail to expose real metrics. FluidWorks is more balanced. It uses visualization as an engineering aid, not as decoration, and it anchors those visuals to diagnostics that can be explained during evaluation.

The AI/ML layer also improves the project in a practical way. Instead of adding a detached machine-learning experiment for the sake of the degree title, the repository embeds a predictor where it is useful: in early geometry assessment and model-quality explanation. That makes the AI/ML inclusion coherent with the rest of the software.

7.3 Future Enhancements

Several extensions would strengthen the platform further:

1. selectable solver presets with clearer trade-offs between speed and quality;
2. richer boundary and material models for internal flow;

3. comparative multi-model analysis inside one project session;
4. broader export formats and automated summary generation;
5. stronger benchmark-based validation against known reference cases; and
6. expansion of the AI/ML layer into setup recommendation or anomaly detection.

Chapter 8

Conclusion & Future Work

FluidWorks has developed into a practical real-time CFD-style visualization platform with a clear architectural center: a maintained wind-tunnel workflow supported by GPU solving, runtime model handling, readable visualization modes, and engineering-focused diagnostics. The repository also shows meaningful progress in internal-flow analysis, especially through explicit boundary assignment and surface-based diagnostics.

The inclusion of AI/ML in the current project is also genuine and relevant. The lightweight geometry-analysis pipeline adds predictive feedback, feature sensitivity, and actionable design suggestions without disrupting the core solver workflow. This makes the software more useful during early exploration and better aligned with the academic domain of the degree program.

In its current state, FluidWorks should be understood as an interactive educational and concept-analysis platform. It sits between classroom theory and heavy-weight offline CFD workflows. That position gives the project both technical value and practical usefulness, and it provides a credible foundation for future expansion in simulation quality, design intelligence, and automated engineering assistance.

Bibliography

- [1] J. D. Anderson, *Computational Fluid Dynamics: The Basics with Applications*. New York, NY, USA: McGraw-Hill, 1995.
- [2] H. K. Versteeg and W. Malalasekera, *An Introduction to Computational Fluid Dynamics: The Finite Volume Method*, 2nd ed. Harlow, U.K.: Pearson, 2007.
- [3] F. M. White, *Fluid Mechanics*, 8th ed. New York, NY, USA: McGraw-Hill, 2016.
- [4] J. H. Ferziger and M. Peric, *Computational Methods for Fluid Dynamics*, 3rd ed. Berlin, Germany: Springer, 2002.
- [5] R. Bridson, *Fluid Simulation for Computer Graphics*, 2nd ed. Boca Raton, FL, USA: CRC Press, 2015.
- [6] J. Stam, “Stable fluids,” in *Proc. 26th Annu. Conf. Computer Graphics and Interactive Techniques (SIGGRAPH)*, 1999, pp. 121–128.
- [7] M. J. Harris, “Fast fluid dynamics simulation on the GPU,” in *GPU Gems*, R. Fernando, Ed. Boston, MA, USA: Addison-Wesley, 2004, ch. 38.
- [8] Unity Technologies, *Unity Manual and Compute Shader Documentation*. [Online]. Available: <https://docs.unity3d.com/>.
- [9] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [10] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [11] C. Molnar, *Interpretable Machine Learning*, 2nd ed. [Online]. Available: <https://christophm.github.io/interpretable-ml-book/>
- [12] M. T. Ribeiro, S. Singh, and C. Guestrin, “Why should I trust you?: Explaining the predictions of any classifier,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.
- [13] S. B. Pope, *Turbulent Flows*. Cambridge, U.K.: Cambridge Univ. Press, 2000.
- [14] S. V. Patankar, *Numerical Heat Transfer and Fluid Flow*. Boca Raton, FL, USA: CRC Press, 1980.
- [15] J. Tu, G. H. Yeoh, and C. Liu, *Computational Fluid Dynamics: A Practical Approach*, 3rd ed. Oxford, U.K.: Butterworth-Heinemann, 2023.

Appendix A

Wind-Tunnel Runtime Driver Excerpt

This appendix provides the opening portion of the wind-tunnel runtime driver, which is the central orchestration point for the maintained external-aero workflow.

```
1 using UnityEngine;
2 using AeroFlow.Visualization;
3 using AeroFlow.UI;
4 using AeroFlow.Display;
5 using AeroFlow.Core;
6
7 [System.Serializable]
8 public enum WindTunnelFlowAxis
9 {
10     LocalX,
11     LocalY,
12     LocalZ
13 }
14
15 [System.Serializable]
16 public enum WindTunnelGraphicsMode
17 {
18     Fluid,
19     Particle,
20     Off
21 }
22
23 [System.Serializable]
24 public class WindTunnelVehicleProperties
25 {
26     public float massKg = 1500f;
27     public float referenceArea = 0f;
28     public float frontWeightDistribution = 0.55f;
29     public float frontAeroBalance = 0.45f;
30     public float wheelbaseMeters = 2.6f;
31     public float cgHeightMeters = 0.35f;
32     public float rideHeightMeters = 0f;
33     public float rakeAngleDegrees = 0f;
34     public float trackWidthMeters = 0f;
35     public float wheelRadiusMeters = 0f;
36     public float wheelWidthMeters = 0f;
37     public float enginePowerKw = 180f;
38     public float drivetrainEfficiency = 0.90f;
39     public float rollingResistanceCoeff = 0.015f;
40     public bool useMovingGround = true;
41     public bool useWheelRotationProxies = true;
42     public float groundSpeedScale = 1f;
43 }
44
45 [System.Serializable]
46 public struct WindTunnelVehicleReferenceFrame
47 {
48     public bool valid;
49     public Bounds modelBounds;
50     public Vector3 flowAxis;
```

```

51     public Vector3 sideAxis;
52     public Vector3 upAxis;
53     public float bodyLength;
54     public float bodyWidth;
55     public float bodyHeight;
56     public float floorCoordinate;
57     public float wheelbase;
58     public float trackWidth;
59     public float wheelRadius;
60     public float wheelWidth;
61     public Vector3 frontAxleCenter;
62     public Vector3 rearAxleCenter;
63     public Vector3 centerOfGravity;
64 }
65
66 [System.Serializable]
67 public class WindTunnelSettings
68 {
69     public string tunnelProfile = "Standard";
70     public bool keepPerformanceBudget = true;
71     public int targetParticleBudget = 15000;
72     public float airDensity = 1.225f;
73     public float dynamicViscosity = 0.0000181f;
74     public float fluidTemperatureC = 20f;
75     public float inletVelocity = 50f;
76     public float outletStaticPressurePa = 101325f;
77     public float angleOfAttack = 0f;
78     public float turbulenceIntensity = 1.0f;
79     public bool useCustomInletDirection = false;
80     public bool flipVehicleDirection = true;
81     public Vector3 inletDirection = new Vector3(1, 0, 0);
82     public string inletSource = "Auto";
83     public string visualizationMode = "Streamlines";
84     public WindTunnelGraphicsMode graphicsMode = WindTunnelGraphicsMode.Fluid;
85     public int streamlineDensity = WindTunnelSimulation3D.DefaultStreamlineDensity;
86     public float timeScale = 1.0f;
87     public int iterationsPerFrame = 3;
88     public WindTunnelVehicleProperties vehicle = new WindTunnelVehicleProperties();
89 }
90
91 public class WindTunnelSimulation3D : MonoBehaviour
92 {
93     public const string PrimarySolverName = "Navier-Stokes Grid (GPU)";
94     public const string StandardProfileName = "Standard";
95     public const string F1BalancedProfileName = "F1 Balanced";
96     public const string VisualizationStreamlines = "Streamlines";
97     public const string VisualizationVelocity = "Velocity";
98     public const string VisualizationPressure = "Pressure";
99     public const string VisualizationEffects = "Effects";
100    public const string VisualizationSmoke = "Smoke";
101    public const string VisualizationOff = "Off";
102    public const string VisualizationVerticalStreamlines = "Vertical Streamlines";
103    public const string VisualizationHorizontalStreamlines = "Horizontal Streamlines";
104    public const string VisualizationSurfaceFriction = "Surface Friction";
105    public const string VisualizationSurfacePressure = "Surface Pressure";
106    public const int MinStreamlineDensity = 40;
107    public const int MaxStreamlineDensity = 280;
108    public const int DefaultStreamlineDensity = 140;
109    public bool showInstancedParticles { get; set; }
110    public WindTunnelSettings settings = new WindTunnelSettings();
111    [Header("Flow Alignment")]
112    public WindTunnelFlowAxis flowAxis = WindTunnelFlowAxis.LocalZ;
113    [Header("Debug")]
114    public bool useRawSolverDataOnly = false;
115    [Header("Demo Wind Tunnel Mode")]
116    public bool demoMode = false;
117    [Range(0f, 10f)] public float windRelaxation = 2.0f;
118    [Range(0f, 2f)] public float noiseStrength = 0.05f;
119    [Header("Legacy Tuning")]
120    public bool fixedTimeStep;
121    [Range(0, 1)] public float collisionDamping = 0.05f;
122    public float smoothingRadius = 0.2f;

```

```

123 public float pressureMultiplier = 150f;
124 public float nearPressureMultiplier = 2.25f;
125 [Header("References")]
126 public Transform floorDisplay;
127 public WindTunnelStreamlineRenderer streamlineRenderer;
128 public AeroFlow.Visualization.StreamlineFieldRenderer streamlineFieldRenderer;
129 public AeroFlow.Visualization.FlowFieldSliceRenderer flowFieldSliceRenderer;
130 public NavierStokesGridSolver navierStokesSolver;
131 public AeroFlow.Physics.SurfaceAeroSolver surfaceAeroSolver;
132
133 bool isPaused = true;
134 bool pauseNextFrame;
135 bool isInitialized;
136 public string initStatus = "Not Initialized";
137 float currentStepTime = 1f / 60f;
138 bool hasPreviewSignature;
139 int lastPreviewSignature;
140 float nextPreviewRefreshTime;
141 string lastNonPressureVisualizationMode = VisualizationStreamlines;
142
143 public void Play()
144 {
145     InitializeIfNeeded();
146     hasPreviewSignature = false;
147     nextPreviewRefreshTime = 0f;
148     isPaused = false;
149 }
150
151 public void Pause()
152 {
153     hasPreviewSignature = false;
154     nextPreviewRefreshTime = 0f;
155     isPaused = true;
156 }
157
158 public bool IsPaused => isPaused;
159
160 void Start()
161 {
162     ClampSettings();
163     SetVisualizationMode(settings.visualizationMode);
164     InitializeIfNeeded();
165     isPaused = true;
166     ApplyFloorColor();
167
168     // Auto-attach the tunnel enclosure visuals if not already present.
169     if (GetComponent<AeroFlow.Visualization.WindTunnelEnclosure>() == null)
170         gameObject.AddComponent<AeroFlow.Visualization.WindTunnelEnclosure>();
171 }
172
173 void OnValidate()
174 {
175     ClampSettings();
176     SetVisualizationMode(settings.visualizationMode);
177     if (streamlineFieldRenderer == null)
178     {
179         streamlineFieldRenderer = GetComponent<AeroFlow.Visualization.StreamlineFieldRenderer>()
180     ;
181     }
182     if (streamlineRenderer == null)
183     {
184         streamlineRenderer = GetComponent<WindTunnelStreamlineRenderer>();
185     }
186     if (flowFieldSliceRenderer == null)
187     {
188         flowFieldSliceRenderer = GetComponent<AeroFlow.Visualization.FlowFieldSliceRenderer>();
189     }
190 }
191
192 void FixedUpdate()
193 {
194     if (fixedTimeStep)

```

```

194     {
195         RunSimulationFrame(Time.fixedDeltaTime);
196     }
197 }
198
199 void Update()
200 {
201     if (!isInitialized) return;
202     ClampSettings();
203     if (!fixedTimeStep && Time.frameCount > 10)
204     {
205         if (isPaused)
206         {
207             RefreshPreviewIfNeeded();
208         }
209         else
210         {
211             RunSimulationFrame(Time.deltaTime);
212         }
213     }
214     else if (fixedTimeStep && isPaused)
215     {
216         RefreshPreviewIfNeeded();
217     }
218
219     if (pauseNextFrame)
220     {
221         isPaused = true;
222         pauseNextFrame = false;
223     }
224
225     ApplyFloorColor();
226     HandleInput();
227 }
228
229 void RunSimulationFrame(float frameTime)
230 {
231     if (!isInitialized || isPaused) return;
232     settings.iterationsPerFrame = Mathf.Max(1, settings.iterationsPerFrame);
233     float clampedFrameTime = Mathf.Min(frameTime, 0.05f);
234     float timeStep = clampedFrameTime / settings.iterationsPerFrame * settings.timeScale;
235     UpdateSettings(timeStep);
236     for (int i = 0; i < settings.iterationsPerFrame; i++)
237     {
238         RunSimulationStep();
239     }
240 }
241
242 void RunSimulationStep()
243 {
244     if (!isInitialized) return;
245     if (navierStokesSolver != null)
246     {
247         GetLoadedModelBounds(out var obstacleCenter, out var obstacleSize);
248         Transform obstacleRoot = GetLoadedModelRoot();
249
250         // Use transform position/scale as wind tunnel bounds, but provide sensible defaults
251         Bounds tunnelBounds = GetTunnelBounds();
252         Vector3 boundsCenter = tunnelBounds.center;
253         Vector3 boundsSize = tunnelBounds.size;
254
255         // If bounds are zero or very small, use default wind tunnel dimensions
256         if (boundsSize.x < 0.01f || boundsSize.y < 0.01f || boundsSize.z < 0.01f)
257         {
258             boundsSize = new Vector3(10f, 5f, 5f); // Default wind tunnel size
259             boundsCenter = transform.position;
260             Debug.Log("[WindTunnel] Using default bounds: " + boundsSize);
261         }
262
263         navierStokesSolver.Step(
264             settings,
265             null,

```

```

266         null,
267         boundsCenter,
268         boundsSize,
269         obstacleCenter,
270         obstacleSize,
271         obstacleRoot,
272         ResolveWindDirection(),
273         ResolveTunnelVerticalAxis(),
274         TryGetVehicleReferenceFrame(out WindTunnelVehicleReferenceFrame vehicleFrame),
275         vehicleFrame,
276         currentStepTime
277     );
278 }
279 }
280
281 void UpdateSettings(float deltaTime)
282 {
283     if (!isInitialized) return;
284     currentStepTime = deltaTime;
285 }
286
287 void RefreshPreviewIfNeeded()
288 {
289     if (!isInitialized) return;
290     if (Time.time < nextPreviewRefreshTime) return;
291
292     int previewSignature = ComputePreviewSignature();
293     if (hasPreviewSignature && previewSignature == lastPreviewSignature)
294     {
295         return;
296     }
297
298     UpdateSettings((1f / 60f) * Mathf.Max(0.1f, settings.timeScale));
299     int previewSteps = Mathf.Clamp(settings.iterationsPerFrame * 3, 8, 16);
300     for (int i = 0; i < previewSteps; i++)
301     {
302         RunSimulationStep();
303     }
304
305     lastPreviewSignature = ComputePreviewSignature();
306     hasPreviewSignature = true;
307     nextPreviewRefreshTime = Time.time + 0.05f;
308 }
309
310 int ComputePreviewSignature()
311 {
312     GetLoadedModelBounds(out var obstacleCenter, out var obstacleSize);
313     Transform obstacleRoot = GetLoadedModelRoot();
314     Vector3 windDirection = ResolveWindDirection();
315
316     int hash = 17;
317     hash = hash * 31 + Quantize(settings.inletVelocity, 100f);
318     hash = hash * 31 + Quantize(settings.angleOfAttack, 100f);
319     hash = hash * 31 + Quantize(settings.turbulenceAttackIntensity, 100f);
320     hash = hash * 31 + Quantize(settings.airDensity, 100f);
321     hash = hash * 31 + Quantize(settings.dynamicViscosity, 100000f);
322     if (settings.vehicle != null)
323     {
324         hash = hash * 31 + Quantize(settings.vehicle.rideHeightMeters, 1000f);
325         hash = hash * 31 + Quantize(settings.vehicle.rakeAngleDegrees, 100f);
326         hash = hash * 31 + Quantize(settings.vehicle.wheelbaseMeters, 100f);
327         hash = hash * 31 + Quantize(settings.vehicle.trackWidthMeters, 100f);
328         hash = hash * 31 + Quantize(settings.vehicle.wheelRadiusMeters, 100f);
329         hash = hash * 31 + Quantize(settings.vehicle.wheelWidthMeters, 100f);
330         hash = hash * 31 + Quantize(settings.vehicle.cgHeightMeters, 100f);
331         hash = hash * 31 + Quantize(settings.vehicle.groundSpeedScale, 100f);
332         hash = hash * 31 + (settings.vehicle.useMovingGround ? 1 : 0);
333         hash = hash * 31 + (settings.vehicle.useWheelRotationProxies ? 1 : 0);
334     }
335     hash = hash * 31 + Quantize(windDirection.x, 1000f);
336     hash = hash * 31 + Quantize(windDirection.y, 1000f);
337     hash = hash * 31 + Quantize(windDirection.z, 1000f);

```

```
338     hash = hash * 31 + Quantize(obstacleCenter.x, 100f);  
339     hash = hash * 31 + Quantize(obstacleCenter.y, 100f);  
340     hash = hash * 31 + Quantize(obstacleCenter.z, 100f);
```

Listing A.1: Excerpt from the wind-tunnel runtime driver

Appendix B

Navier-Stokes Grid Solver Excerpt

The following excerpt grounds the report’s discussion of solver-side grid handling, diagnostics, and obstacle coupling.

```
1 using UnityEngine;
2 using System.Collections.Generic;
3 using System;
4
5 public class NavierStokesGridSolver : MonoBehaviour
6 {
7     public event Action OnSimulationCompleted;
8
9     public struct NavierDiagnostics
10    {
11        public bool valid;
12        public float meanVelocity;
13        public float maxVelocity;
14        public float pressureDrop;
15        public float wallShear;
16        public float divergenceL1;
17        public float meanVorticity;
18    }
19
20    [Header("Grid Resolution")]
21    [Range(16, 128)] public int gridSizeX = 56;
22    [Range(8, 96)] public int gridSizeY = 28;
23    [Range(8, 96)] public int gridSizeZ = 28;
24
25    [Header("Solver Quality")]
26    [Range(1, 12)] public int diffusionIterations = 2;
27    [Range(4, 80)] public int pressureIterations = 14;
28    [Range(0f, 0.05f)] public float velocityDamping = 0.002f;
29    [Range(0f, 3f)] public float vorticityConfinement = 0.6f;
30
31    public ComputeShader navierShader;
32
33    ComputeBuffer gridVelocity;
34    ComputeBuffer gridVelocityTmp;
35    ComputeBuffer gridPressure;
36    ComputeBuffer gridPressureTmp;
37    ComputeBuffer gridDivergence;
38    ComputeBuffer obstacleSphereBuffer;
39    ComputeBuffer obstacleMaskBuffer;
40    ComputeBuffer obstacleVelocityBuffer;
41
42    int applyForcesKernel;
43    int advectVelocityKernel;
44    int diffuseKernel;
45    int vorticityKernel;
46    int divergenceKernel;
47    int jacobiKernel;
48    int projectKernel;
49    int advectParticlesKernel;
50
```

```

51     bool initialized;
52     int initializedGridCount;
53     Vector3 lastCellSize = Vector3.one;
54     NavierDiagnostics diagnostics;
55     float nextDiagnosticsTime;
56     float nextObstacleProxyUpdateTime;
57     float lastInletVelocity;
58     float lastFluidDensity = 1.225f;
59     Vector3 lastWindDirection = Vector3.right;
60     const int MaxObstacleSpheres = 64;
61     readonly Vector4[] obstacleSphereData = new Vector4[MaxObstacleSpheres];
62     static readonly List<Renderer> obstacleRenderers = new List<Renderer>(128);
63     int obstacleSphereCount;
64     int obstacleMaskSolidCount;
65     int[] obstacleMaskData;
66     Vector3[] diagnosticsVelocityCache;
67     float[] diagnosticsPressureCache;
68     float[] diagnosticsDivergenceCache;
69     int diagnosticsSnapshotVersion;
70     Vector3 lastBoundsCenter;
71     Vector3 lastBoundsSize;
72     Vector3[] velocitySnapshot;
73     int lastLoggedObstacleSphereCount = -1;
74     int lastLoggedObstacleMaskSolidCount = -1;
75     float nextObstacleLogTime;
76
77     public Vector3 BoundsCenter => lastBoundsCenter;
78     public Vector3 BoundsSize => lastBoundsSize;
79     public Vector3 CellSize => lastCellSize;
80     public int ObstacleSphereCount => obstacleSphereCount;
81     public int ObstacleMaskSolidCount => obstacleMaskSolidCount;
82
83     public bool TryGetDiagnostics(out NavierDiagnostics value)
84     {
85         value = diagnostics;
86         return diagnostics.valid;
87     }
88
89     public bool TryGetPressureRange(out float minPressure, out float maxPressure)
90     {
91         minPressure = 0f;
92         maxPressure = 0f;
93
94         if (!diagnostics.valid || diagnosticsPressureCache == null)
95         {
96             return false;
97         }
98
99         int count = diagnosticsPressureCache.Length;
100         if (count == 0)
101         {
102             return false;
103         }
104
105         minPressure = float.PositiveInfinity;
106         maxPressure = float.NegativeInfinity;
107         for (int i = 0; i < count; i++)
108         {
109             float p = diagnosticsPressureCache[i];
110             if (!float.IsFinite(p))
111             {
112                 continue;
113             }
114
115             if (p < minPressure) minPressure = p;
116             if (p > maxPressure) maxPressure = p;
117         }
118
119         if (!float.IsFinite(minPressure) || !float.IsFinite(maxPressure))
120         {
121             minPressure = 0f;
122             maxPressure = 0f;

```



```

123         return false;
124     }
125
126     return true;
127 }
128
129 public bool IsObstacle(Vector3 worldPos)
130 {
131     if (obstacleMaskData == null || obstacleMaskSolidCount <= 0) return false;
132     int gridCount = gridSizeX * gridSizeY * gridSizeZ;
133     if (obstacleMaskData.Length != gridCount) return false;
134     if (lastBoundsSize.x <= 1e-5f || lastBoundsSize.y <= 1e-5f || lastBoundsSize.z <= 1e-5f)
135         return false;
136
137     Vector3 minB = lastBoundsCenter - lastBoundsSize * 0.5f;
138     float ux = (worldPos.x - minB.x) / lastBoundsSize.x;
139     float uy = (worldPos.y - minB.y) / lastBoundsSize.y;
140     float uz = (worldPos.z - minB.z) / lastBoundsSize.z;
141     if (ux < 0f || ux > 1f || uy < 0f || uy > 1f || uz < 0f || uz > 1f) return false;
142
143     int ix = Mathf.Clamp(Mathf.FloorToInt(ux * (gridSizeX - 1)), 0, gridSizeX - 1);
144     int iy = Mathf.Clamp(Mathf.FloorToInt(uy * (gridSizeY - 1)), 0, gridSizeY - 1);
145     int iz = Mathf.Clamp(Mathf.FloorToInt(uz * (gridSizeZ - 1)), 0, gridSizeZ - 1);
146     return obstacleMaskData[Flatten(ix, iy, iz)] != 0;
147 }
148
149 public void ResetGrid()
150 {
151     ReleaseBuffers();
152     initialized = false;
153 }
154
155 public void Step(
156     WindTunnelSettings settings,
157     ComputeBuffer particlePositions,
158     ComputeBuffer particleVelocities,
159     Vector3 boundsCenter,
160     Vector3 boundsSize,
161     Vector3 obstacleCenter,
162     Vector3 obstacleSize,
163     Transform obstacleRoot,
164     Vector3 windDirection,
165     Vector3 tunnelUpAxis,
166     bool hasVehicleFrame,
167     WindTunnelVehicleReferenceFrame vehicleFrame,
168     float deltaTime)
169 {
170     EnsureInitialized();
171     if (!initialized) return;
172     int particleCount = (particlePositions != null && particleVelocities != null) ?
173         particlePositions.count : 0;
174
175     int gridCount = gridSizeX * gridSizeY * gridSizeZ;
176     windDirection = windDirection.sqrMagnitude > 1e-6f ? windDirection.normalized : Vector3.
177     right;
178     lastWindDirection = windDirection;
179     lastInletVelocity = Mathf.Max(settings.inletVelocity, 0f);
180     lastFluidDensity = Mathf.Max(settings.airDensity, 1e-4f);
181
182     Vector3 cellSize = new Vector3(
183         Mathf.Max(boundsSize.x / gridSizeX, 1e-3f),
184         Mathf.Max(boundsSize.y / gridSizeY, 1e-3f),
185         Mathf.Max(boundsSize.z / gridSizeZ, 1e-3f)
186     );
187     float minCell = Mathf.Min(cellSize.x, Mathf.Min(cellSize.y, cellSize.z));
188     float cappedDt = Mathf.Max(0.0012f, minCell / Mathf.Max(lastInletVelocity, 1f) * 1.2f);
189     float safeDt = Mathf.Clamp(Mathf.Min(deltaTime, cappedDt), 1e-4f, 0.05f);
190     lastCellSize = cellSize;
191
192     navierShader.SetInt("gridSizeX", gridSizeX);
193     navierShader.SetInt("gridSizeY", gridSizeY);
194     navierShader.SetInt("gridSizeZ", gridSizeZ);

```

```

192     navierShader.SetInt("gridCount", gridCount);
193     navierShader.SetInt("particleCount", particleCount);
194     navierShader.SetFloat("deltaTime", safeDt);
195     navierShader.SetFloat("viscosity", Mathf.Max(settings.dynamicViscosity, 1e-6f));
196     navierShader.SetFloat("velocityDamping", velocityDamping);
197     navierShader.SetFloat("inletVelocity", Mathf.Max(settings.inletVelocity, 0f));
198     navierShader.SetFloat("turbulenceIntensity", Mathf.Clamp01(settings.turbulenceIntensity *
0.01f));
199     navierShader.SetFloat("vorticityStrength", Mathf.Max(0f, vorticityConfinement));
200     navierShader.SetVector("windDirection", windDirection);
201     navierShader.SetVector("tunnelUpDirection", tunnelUpAxis.sqrMagnitude > 1e-6f ? tunnelUpAxis
.normalized : Vector3.up);
202     navierShader.SetVector("boundsCenter", boundsCenter);
203     navierShader.SetVector("boundsSize", boundsSize);
204     lastBoundsCenter = boundsCenter;
205     lastBoundsSize = boundsSize;
206     navierShader.SetVector("cellSize", cellSize);
207     navierShader.SetVector("obstacleCenter", obstacleCenter);
208     navierShader.SetVector("obstacleSize", obstacleSize);
209     UpdateObstacleProxyData(obstacleRoot, obstacleCenter, obstacleSize);
210     navierShader.SetInt("obstacleSphereCount", obstacleSphereCount);
211     navierShader.SetInt("useMovingGround", settings.vehicle != null && settings.vehicle.
useMovingGround ? 1 : 0);
212     navierShader.SetInt("useWheelRotationProxies", hasVehicleFrame && settings.vehicle != null
&& settings.vehicle.useWheelRotationProxies ? 1 : 0);
213     navierShader.SetFloat("groundSpeedScale", settings.vehicle != null ? settings.vehicle.
groundSpeedScale : 1f);
214     navierShader.SetVector("frontAxleCenter", hasVehicleFrame ? vehicleFrame.frontAxleCenter :
Vector3.zero);
215     navierShader.SetVector("rearAxleCenter", hasVehicleFrame ? vehicleFrame.rearAxleCenter :
Vector3.zero);
216     navierShader.SetFloat("wheelTrack", hasVehicleFrame ? vehicleFrame.trackWidth : 0f);
217     navierShader.SetFloat("wheelRadius", hasVehicleFrame ? vehicleFrame.wheelRadius : 0f);
218     navierShader.SetFloat("wheelWidth", hasVehicleFrame ? vehicleFrame.wheelWidth : 0f);
219     navierShader.SetFloat("timeValue", Time.time);
220     if (particleCount > 0)
221     {
222         navierShader.SetBuffer(advectParticlesKernel, "ParticlePositions", particlePositions);
223         navierShader.SetBuffer(advectParticlesKernel, "ParticleVelocities", particleVelocities);
224     }
225
226     ComputeHelper.Dispatch(navierShader, gridSizeX, gridSizeY, gridSizeZ, applyForcesKernel);
227     ComputeHelper.Dispatch(navierShader, gridSizeX, gridSizeY, gridSizeZ, advectVelocityKernel);
228     Swap(ref gridVelocity, ref gridVelocityTmp);
229     BindBuffers();
230
231     for (int i = 0; i < diffusionIterations; i++)
232     {
233         ComputeHelper.Dispatch(navierShader, gridSizeX, gridSizeY, gridSizeZ, diffuseKernel);
234         Swap(ref gridVelocity, ref gridVelocityTmp);
235         BindBuffers();
236     }
237
238     if (vorticityConfinement > 1e-4f)
239     {
240         ComputeHelper.Dispatch(navierShader, gridSizeX, gridSizeY, gridSizeZ, vorticityKernel);
241         Swap(ref gridVelocity, ref gridVelocityTmp);
242         BindBuffers();
243     }
244
245     ComputeHelper.Dispatch(navierShader, gridSizeX, gridSizeY, gridSizeZ, divergenceKernel);
246
247     for (int i = 0; i < pressureIterations; i++)
248     {
249         ComputeHelper.Dispatch(navierShader, gridSizeX, gridSizeY, gridSizeZ, jacobiKernel);
250         Swap(ref gridPressure, ref gridPressureTmp);
251         BindBuffers();
252     }
253
254     ComputeHelper.Dispatch(navierShader, gridSizeX, gridSizeY, gridSizeZ, projectKernel);
255     if (particleCount > 0)
256     {

```

```
257     ComputeHelper.Dispatch(navierShader, particleCount, 1, 1, advectParticlesKernel);
258 }
259
260 bool shouldRefreshDiagnostics = !diagnostics.valid
261     || Time.time >= nextDiagnosticsTime
262     || OnSimulationCompleted != null;
263
264 if (shouldRefreshDiagnostics)
265 {
266     UpdateDiagnostics(Mathf.Max(settings.dynamicViscosity, 1e-6f));
267     nextDiagnosticsTime = Time.time + 0.35f;
268 }
269
270 if (diagnostics.valid)
271 {
272     OnSimulationCompleted?.Invoke();
273 }
274 }
275
276 void EnsureInitialized()
277 {
278     int gridCount = gridSizeX * gridSizeY * gridSizeZ;
279     bool needsReinit = !initialized || gridCount != initializedGridCount;
280     if (!needsReinit) return;
281
282     if (navierShader == null)
283     {
284         navierShader = Resources.Load<ComputeShader>("Compute/WindTunnel/NavierStokes3D");
285         if (navierShader == null)
286         {
287             Debug.LogError("[NavierStokes] Missing compute shader at Resources/Compute/
WindTunnel/NavierStokes3D.compute");
288             return;
289         }
290     }
291
292     ReleaseBuffers();
293     gridVelocity = ComputeHelper.CreateStructuredBuffer<Vector3>(gridCount);
294     gridVelocityTmp = ComputeHelper.CreateStructuredBuffer<Vector3>(gridCount);
295     gridPressure = ComputeHelper.CreateStructuredBuffer<float>(gridCount);
296     gridPressureTmp = ComputeHelper.CreateStructuredBuffer<float>(gridCount);
297     gridDivergence = ComputeHelper.CreateStructuredBuffer<float>(gridCount);
298     obstacleSphereBuffer = ComputeHelper.CreateStructuredBuffer<Vector4>(MaxObstacleSpheres);
299     obstacleMaskBuffer = ComputeHelper.CreateStructuredBuffer<int>(gridCount);
300     obstacleVelocityBuffer = ComputeHelper.CreateStructuredBuffer<Vector3>(gridCount);
```

Listing B.1: Excerpt from the grid solver implementation

Appendix C

Compute Shader Excerpt

This listing shows a representative section of the compute-shader implementation used by the wind-tunnel solver path.

```
1  #pragma kernel ApplyForces
2  #pragma kernel AdvectVelocity
3  #pragma kernel DiffuseVelocity
4  #pragma kernel VorticityConfinement
5  #pragma kernel ComputeDivergence
6  #pragma kernel JacobiPressure
7  #pragma kernel ProjectVelocity
8  #pragma kernel AdvectParticles
9
10 RWStructuredBuffer<float3> GridVelocity;
11 RWStructuredBuffer<float3> GridVelocityTmp;
12 RWStructuredBuffer<float> GridPressure;
13 RWStructuredBuffer<float> GridPressureTmp;
14 RWStructuredBuffer<float> GridDivergence;
15
16 RWStructuredBuffer<float3> ParticlePositions;
17 RWStructuredBuffer<float3> ParticleVelocities;
18 StructuredBuffer<float4> ObstacleSpheres; // xyz center, w radius
19 StructuredBuffer<int> ObstacleMask;
20 StructuredBuffer<float3> ObstacleVelocity;
21
22 int gridSizeX;
23 int gridSizeY;
24 int gridSizeZ;
25 int gridCount;
26 int particleCount;
27
28 float deltaTime;
29 float viscosity;
30 float velocityDamping;
31 float inletVelocity;
32 float turbulenceIntensity;
33 float vorticityStrength;
34 float3 windDirection;
35 float3 boundsCenter;
36 float3 boundsSize;
37 float3 cellSize;
38 float3 obstacleCenter;
39 float3 obstacleSize;
40 int obstacleSphereCount;
41 float timeValue;
42 float3 tunnelUpDirection;
43 int useMovingGround;
44 int useWheelRotationProxies;
45 float groundSpeedScale;
46 float3 frontAxleCenter;
47 float3 rearAxleCenter;
48 float wheelTrack;
49 float wheelRadius;
50 float wheelWidth;
```

```

51
52 static const float EPS = 1e-6;
53
54 float Max3(float3 v)
55 {
56     return max(v.x, max(v.y, v.z));
57 }
58
59 float3 SafeNormalize(float3 v, float3 fallback)
60 {
61     float lenSq = dot(v, v);
62     return lenSq > EPS ? v * rsqrt(lenSq) : fallback;
63 }
64
65 int Flatten(int x, int y, int z)
66 {
67     return x + gridSizeX * (y + gridSizeY * z);
68 }
69
70 int3 ClampCell(int3 c)
71 {
72     return int3(
73         clamp(c.x, 0, gridSizeX - 1),
74         clamp(c.y, 0, gridSizeY - 1),
75         clamp(c.z, 0, gridSizeZ - 1)
76     );
77 }
78
79 float3 CellCenterWorld(int3 c)
80 {
81     float3 uv = (float3(c) + 0.5) / float3(gridSizeX, gridSizeY, gridSizeZ);
82     return boundsCenter + (uv - 0.5) * boundsSize;
83 }
84
85 float3 SampleVelocityAtWorld(float3 world)
86 {
87     float3 minB = boundsCenter - boundsSize * 0.5;
88     float3 uv = saturate((world - minB) / max(boundsSize, float3(1e-3, 1e-3, 1e-3)));
89
90     float fx = clamp(uv.x * (gridSizeX - 1), 0.0, gridSizeX - 1.001);
91     float fy = clamp(uv.y * (gridSizeY - 1), 0.0, gridSizeY - 1.001);
92     float fz = clamp(uv.z * (gridSizeZ - 1), 0.0, gridSizeZ - 1.001);
93
94     int x0 = (int)floor(fx);
95     int y0 = (int)floor(fy);
96     int z0 = (int)floor(fz);
97     int x1 = min(x0 + 1, gridSizeX - 1);
98     int y1 = min(y0 + 1, gridSizeY - 1);
99     int z1 = min(z0 + 1, gridSizeZ - 1);
100
101     float tx = fx - x0;
102     float ty = fy - y0;
103     float tz = fz - z0;
104
105     float3 v000 = GridVelocity[Flatten(x0, y0, z0)];
106     float3 v100 = GridVelocity[Flatten(x1, y0, z0)];
107     float3 v010 = GridVelocity[Flatten(x0, y1, z0)];
108     float3 v110 = GridVelocity[Flatten(x1, y1, z0)];
109     float3 v001 = GridVelocity[Flatten(x0, y0, z1)];
110     float3 v101 = GridVelocity[Flatten(x1, y0, z1)];
111     float3 v011 = GridVelocity[Flatten(x0, y1, z1)];
112     float3 v111 = GridVelocity[Flatten(x1, y1, z1)];
113
114     float3 v00 = lerp(v000, v100, tx);
115     float3 v10 = lerp(v010, v110, tx);
116     float3 v01 = lerp(v001, v101, tx);
117     float3 v11 = lerp(v011, v111, tx);
118
119     return lerp(lerp(v00, v10, ty), lerp(v01, v11, ty), tz);
120 }
121
122 float3 SampleVelocityAtCell(int3 c)

```

```

123 {
124     int3 cc = ClampCell(c);
125     return GridVelocity[Flatten(cc.x, cc.y, cc.z)];
126 }
127
128 float3 ComputeCurl(int3 c)
129 {
130     float inv2Dx = 1.0 / (2.0 * max(cellSize.x, 1e-3));
131     float inv2Dy = 1.0 / (2.0 * max(cellSize.y, 1e-3));
132     float inv2Dz = 1.0 / (2.0 * max(cellSize.z, 1e-3));
133
134     float3 vxp = SampleVelocityAtCell(c + int3(1, 0, 0));
135     float3 vxm = SampleVelocityAtCell(c + int3(-1, 0, 0));
136     float3 vyp = SampleVelocityAtCell(c + int3(0, 1, 0));
137     float3 vym = SampleVelocityAtCell(c + int3(0, -1, 0));
138     float3 vzp = SampleVelocityAtCell(c + int3(0, 0, 1));
139     float3 vzm = SampleVelocityAtCell(c + int3(0, 0, -1));
140
141     return float3(
142         (vyp.z - vym.z) * inv2Dy - (vzp.y - vzm.y) * inv2Dz,
143         (vzp.x - vzm.x) * inv2Dz - (vxp.z - vxm.z) * inv2Dx,
144         (vxp.y - vxm.y) * inv2Dx - (vyp.x - vym.x) * inv2Dy);
145 }
146
147 void BuildObstacleBasis(float3 dir, out float3 side, out float3 up)
148 {
149     float3 referenceUp = SafeNormalize(tunnelUpDirection, float3(0, 1, 0));
150     if (abs(dot(referenceUp, dir)) > 0.95)
151     {
152         referenceUp = abs(dir.y) < 0.95 ? float3(0, 1, 0) : float3(1, 0, 0);
153     }
154
155     side = SafeNormalize(cross(referenceUp, dir), float3(1, 0, 0));
156     up = SafeNormalize(cross(dir, side), referenceUp);
157 }
158
159 float ProjectHalfExtent(float3 halfExtents, float3 axis)
160 {
161     return dot(halfExtents, abs(axis));
162 }
163
164 float3 ComputeObstacleFlowField(float3 world, float3 dir, float3 baseVelocity)
165 {
166     float3 halfExtents = obstacleSize * 0.5;
167     if (halfExtents.x <= EPS || halfExtents.y <= EPS || halfExtents.z <= EPS)
168     {
169         return baseVelocity;
170     }
171
172     float3 side;
173     float3 up;
174     BuildObstacleBasis(dir, side, up);
175
176     float minScale = Max3(cellSize) * 1.6;
177     float halfLength = max(ProjectHalfExtent(halfExtents, dir) * 1.12, minScale);
178     float halfWidth = max(ProjectHalfExtent(halfExtents, side) * 0.92, minScale * 0.75);
179     float halfHeight = max(ProjectHalfExtent(halfExtents, up) * 0.92, minScale * 0.75);
180
181     float3 rel = world - obstacleCenter;
182     float axial = dot(rel, dir);
183     float lateral = dot(rel, side);
184     float vertical = dot(rel, up);
185
186     float ellipsoid = axial * axial / max(halfLength * halfLength, 1e-4)
187         + lateral * lateral / max(halfWidth * halfWidth, 1e-4)
188         + vertical * vertical / max(halfHeight * halfHeight, 1e-4);
189
190     float3 lateralDir = side * (lateral / max(halfWidth * halfWidth, 1e-4))
191         + up * (vertical / max(halfHeight * halfHeight, 1e-4));
192     lateralDir = SafeNormalize(lateralDir, side);
193
194     if (ellipsoid < 1.0)

```

```
195 {
196     float3 normal = SafeNormalize(dir * (axial / max(halfLength * halfLength, 1e-4)) +
    lateralDir, dir);
197     float3 tangent = dir - normal * dot(dir, normal);
198     tangent = SafeNormalize(tangent, dir);
199     return tangent * (inletVelocity * 0.20) + normal * (inletVelocity * 0.02);
200 }
201
202 float shell = sqrt(max(ellipsoid, 1e-4));
203 float influence = saturate(2.15 - shell);
204 float throat = saturate(1.28 - abs(axial) / max(halfLength * 1.8, 1e-4));
205 float crossRadius = sqrt(
206     lateral * lateral / max(halfWidth * halfWidth, 1e-4)
207     + vertical * vertical / max(halfHeight * halfHeight, 1e-4));
208 float nearSurface = saturate(1.55 - crossRadius);
209
210 float3 sidewash = lateralDir * (inletVelocity * 0.95 * influence);
211 float3 acceleratedCore = dir * (inletVelocity * 0.30 * throat * nearSurface);
212
213 float3 wake = float3(0, 0, 0);
214 if (axial > 0.0)
215 {
216     float wakeLength = max(halfLength * 8.0, minScale * 6.0);
217     float wakeAxial = saturate(1.0 - axial / wakeLength);
218     float wakeCore = wakeAxial * saturate(1.30 - crossRadius);
219     wake -= dir * (inletVelocity * 0.88 * wakeCore);
220 }
```

Listing C.1: Excerpt from the wind-tunnel compute shader

Appendix D

AI/ML Predictor Excerpt

This appendix captures the lightweight multilayer perceptron used for geometry-based aerodynamic prediction.

```
1 using UnityEngine;
2
3 namespace AeroFlow.Physics
4 {
5     /// <summary>
6     /// Lightweight MLP (Multi-Layer Perceptron) neural network for aerodynamic prediction.
7     /// Uses physics-informed pre-trained weights - no external dependencies.
8     /// Architecture: 8 inputs -> 16 hidden (ReLU) -> 8 hidden (ReLU) -> 4 outputs (Sigmoid).
9     ///
10    /// Input features (from AeroGeometryAnalyzer):
11    /// [0] aspectRatio, [1] slendernessRatio, [2] frontalBlockageRatio, [3] volumeEfficiency,
12    /// [4] surfaceRoughness, [5] symmetryScore, [6] undersideClearance, [7] rearTaperRatio
13    ///
14    /// Outputs:
15    /// [0] predictedCd, [1] separationRisk, [2] downforcePotential, [3] efficiencyScore
16    /// </summary>
17    public static class AeroMLPredictor
18    {
19        public struct PredictionResult
20        {
21            public float predictedCd;
22            public float separationRisk;
23            public float downforcePotential;
24            public float efficiencyScore;
25        }
26
27        private const int InputSize = 8;
28        private const int Hidden1Size = 16;
29        private const int Hidden2Size = 8;
30        private const int OutputSize = 4;
31
32        /// Physics-informed weights encode known aerodynamic relationships:
33        /// - Higher frontal area -> higher Cd
34        /// - Better slenderness -> lower drag
35        /// - Surface roughness -> promotes separation
36        /// - Rear taper -> reduces wake drag
37        /// - Symmetry -> reduces side force / improves stability
38
39        /// Layer 1: Input(8) -> Hidden1(16), weights[16][8]
40        private static readonly float[,] W1 = {
41            /// Each row = one hidden neuron's connections to all 8 inputs
42            /// aspectR slender blockage volEff rough symmetry clearance rearTaper
43            { 0.25f, 0.30f, -0.45f, 0.15f, -0.35f, 0.10f, 0.05f, 0.28f }, // drag-low
44            { -0.20f, -0.25f, 0.50f, -0.10f, 0.40f, -0.05f, 0.02f, -0.30f }, // drag-high
45            { 0.10f, 0.15f, -0.20f, 0.25f, -0.50f, 0.08f, 0.03f, 0.35f }, // smooth
46            { -0.05f, -0.10f, 0.35f, -0.15f, 0.55f, -0.12f, -0.02f, -0.25f }, // turbulent
47        }
48    }
49 }
```



```

47         { 0.02f, 0.05f, -0.10f, 0.30f, -0.20f, 0.40f, 0.10f, 0.15f }, // stability
48         { -0.08f, -0.03f, 0.15f, -0.25f, 0.30f, -0.35f, -0.05f, -0.10f }, //
instability
49         { 0.15f, 0.20f, -0.30f, 0.10f, -0.15f, 0.05f, 0.45f, 0.20f }, // ground
effect
50         { -0.12f, 0.08f, 0.05f, 0.20f, -0.25f, 0.15f, -0.40f, 0.10f }, // clearance
effect
51         { 0.30f, 0.35f, -0.15f, 0.05f, -0.10f, 0.20f, 0.08f, 0.40f }, //
efficiency high
52         { -0.28f, -0.32f, 0.20f, -0.08f, 0.12f, -0.18f, -0.06f, -0.38f }, //
efficiency low
53         { 0.05f, 0.12f, -0.25f, 0.35f, -0.45f, 0.10f, 0.15f, 0.22f }, // low
separation
54         { -0.03f, -0.08f, 0.30f, -0.30f, 0.48f, -0.08f, -0.10f, -0.18f }, // high
separation
55         { 0.18f, 0.10f, -0.05f, 0.08f, -0.12f, 0.25f, 0.30f, 0.08f }, // downforce
potential
56         { -0.15f, 0.05f, 0.10f, 0.12f, -0.08f, 0.08f, -0.25f, 0.15f }, // lift
tendency
57         { 0.22f, 0.28f, -0.35f, 0.18f, -0.30f, 0.15f, 0.12f, 0.32f }, // overall
quality
58         { -0.18f, -0.22f, 0.28f, -0.12f, 0.25f, -0.10f, -0.08f, -0.28f } // overall
penalty
59     };
60     private static readonly float[] B1 = {
61         0.10f, -0.05f, 0.08f, -0.03f, 0.12f, -0.08f, 0.05f, 0.02f,
62         0.15f, -0.10f, 0.06f, -0.04f, 0.08f, 0.03f, 0.12f, -0.07f
63     };
64
65     // Layer 2: Hidden1(16) -> Hidden2(8), weights[8][16]
66     private static readonly float[,] W2 = {
67         { 0.25f, -0.30f, 0.20f, -0.15f, 0.10f, -0.05f, 0.08f, -0.12f, 0.22f, -0.20f, 0.18f
, -0.15f, 0.05f, -0.08f, 0.28f, -0.25f },
68         { -0.20f, 0.35f, -0.15f, 0.25f, -0.08f, 0.12f, -0.05f, 0.10f, -0.18f, 0.22f, -0.12f
, 0.20f, -0.03f, 0.06f, -0.22f, 0.28f },
69         { 0.15f, -0.10f, 0.25f, -0.20f, 0.30f, -0.25f, 0.12f, -0.08f, 0.15f, -0.12f, 0.28f
, -0.22f, 0.10f, -0.05f, 0.20f, -0.18f },
70         { -0.12f, 0.18f, -0.22f, 0.28f, -0.25f, 0.30f, -0.10f, 0.15f, -0.12f, 0.15f, -0.25f
, 0.28f, -0.08f, 0.10f, -0.18f, 0.22f },
71         { 0.10f, -0.08f, 0.12f, -0.10f, 0.15f, -0.12f, 0.35f, -0.30f, 0.08f, -0.05f, 0.10f
, -0.08f, 0.28f, -0.22f, 0.12f, -0.10f },
72         { -0.08f, 0.12f, -0.10f, 0.15f, -0.12f, 0.18f, -0.30f, 0.35f, -0.06f, 0.08f, -0.08f
, 0.12f, -0.25f, 0.28f, -0.10f, 0.12f },
73         { 0.20f, -0.15f, 0.18f, -0.12f, 0.08f, -0.05f, 0.10f, -0.08f, 0.30f, -0.28f, 0.15f
, -0.10f, 0.12f, -0.05f, 0.25f, -0.22f },
74         { -0.18f, 0.20f, -0.15f, 0.18f, -0.10f, 0.08f, -0.08f, 0.12f, -0.28f, 0.30f, -0.12f
, 0.15f, -0.10f, 0.08f, -0.22f, 0.25f }
75     };
76     private static readonly float[] B2 = {
77         0.08f, -0.05f, 0.06f, -0.04f, 0.10f, -0.06f, 0.07f, -0.03f
78     };
79
80     // Layer 3: Hidden2(8) -> Output(4), weights[4][8]
81     private static readonly float[,] W3 = {
82         // Cd prediction: favors neurons that detect drag characteristics
83         { -0.35f, 0.40f, -0.20f, 0.25f, -0.10f, 0.15f, -0.30f, 0.35f },
84         // Separation risk: favors turbulence/roughness neurons
85         { -0.15f, 0.20f, -0.30f, 0.35f, -0.10f, 0.12f, -0.18f, 0.22f },
86         // Downforce potential: favors ground effect and clearance neurons
87         { 0.20f, -0.15f, 0.10f, -0.08f, 0.35f, -0.30f, 0.25f, -0.20f },
88         // Efficiency score: overall quality aggregation
89         { 0.30f, -0.25f, 0.28f, -0.22f, 0.15f, -0.10f, 0.32f, -0.28f }
90     };
91     private static readonly float[] B3 = { 0.35f, 0.30f, 0.25f, 0.45f };
92
93     // Feature normalization parameters (mean, scale)
94     private static readonly float[] FeatureMean = { 2.5f, 2.0f, 0.15f, 0.50f, 0.30f, 0.80f,
0.10f, 0.40f };
95     private static readonly float[] FeatureScale = { 2.0f, 1.5f, 0.20f, 0.30f, 0.25f, 0.15f,
0.10f, 0.30f };
96
97     public static PredictionResult Predict(float[] features)

```

```

98     {
99         if (features == null || features.Length != InputSize)
100         {
101             return new PredictionResult
102             {
103                 predictedCd = 0.4f,
104                 separationRisk = 0.5f,
105                 downforcePotential = 0.3f,
106                 efficiencyScore = 0.5f
107             };
108         }
109
110         // Normalize inputs
111         float[] normalized = new float[InputSize];
112         for (int i = 0; i < InputSize; i++)
113         {
114             normalized[i] = (features[i] - FeatureMean[i]) / Mathf.Max(FeatureScale[i], 1e-5f);
115         }
116
117         // Forward pass: Layer 1
118         float[] h1 = new float[Hidden1Size];
119         for (int j = 0; j < Hidden1Size; j++)
120         {
121             float sum = B1[j];
122             for (int i = 0; i < InputSize; i++)
123                 sum += W1[j, i] * normalized[i];
124             h1[j] = ReLU(sum);
125         }
126
127         // Forward pass: Layer 2
128         float[] h2 = new float[Hidden2Size];
129         for (int j = 0; j < Hidden2Size; j++)
130         {
131             float sum = B2[j];
132             for (int i = 0; i < Hidden1Size; i++)
133                 sum += W2[j, i] * h1[i];
134             h2[j] = ReLU(sum);
135         }
136
137         // Forward pass: Output layer
138         float[] output = new float[OutputSize];
139         for (int j = 0; j < OutputSize; j++)
140         {
141             float sum = B3[j];
142             for (int i = 0; i < Hidden2Size; i++)
143                 sum += W3[j, i] * h2[i];
144             output[j] = Sigmoid(sum);
145         }
146
147         return new PredictionResult
148         {
149             predictedCd = Mathf.Lerp(0.15f, 0.90f, output[0]),
150             separationRisk = output[1],
151             downforcePotential = output[2],
152             efficiencyScore = output[3]
153         };
154     }
155
156     /// <summary>
157     /// Compute feature importance using central-difference gradient approximation.
158     /// Aggregates sensitivity across all 4 outputs for a holistic importance measure.
159     /// Returns normalized importance values (sum to 1.0) for each input feature.
160     /// </summary>
161     public static float[] ComputeFeatureImportance(float[] features)
162     {
163         if (features == null || features.Length != InputSize)
164             return new float[InputSize];
165
166         float[] importance = new float[InputSize];
167         float epsilon = 0.05f;
168
169         for (int i = 0; i < InputSize; i++)

```

```
170     {
171         float[] perturbedPlus = (float[])features.Clone();
172         float[] perturbedMinus = (float[])features.Clone();
173         perturbedPlus[i] += epsilon;
174         perturbedMinus[i] -= epsilon;
175         PredictionResult pPlus = Predict(perturbedPlus);
176         PredictionResult pMinus = Predict(perturbedMinus);
177
178         // Aggregate sensitivity across all outputs using central difference
179         float sensitivity =
180             Mathf.Abs(pPlus.predictedCd - pMinus.predictedCd) +
181             Mathf.Abs(pPlus.separationRisk - pMinus.separationRisk) +
182             Mathf.Abs(pPlus.downforcePotential - pMinus.downforcePotential) +
183             Mathf.Abs(pPlus.efficiencyScore - pMinus.efficiencyScore);
184
185         importance[i] = sensitivity / (2f * epsilon);
186     }
187
188     // Normalize to sum to 1
```

Listing D.1: AeroMLPredictor implementation

Appendix E

Geometry Analysis Excerpt

The geometry-analysis excerpt shows how the project converts model shape into interpretable features and model-improvement suggestions.

```
1 using System.Collections.Generic;
2 using System.Text;
3 using AeroFlow.Core;
4 using UnityEngine;
5
6 namespace AeroFlow.Physics
7 {
8     public class AeroGeometryAnalyzer : MonoBehaviour
9     {
10         public struct GeometricFeatures
11         {
12             public float aspectRatio;
13             public float slendernessRatio;
14             public float frontalBlockageRatio;
15             public float volumeEfficiency;
16             public float surfaceRoughness;
17             public float symmetryScore;
18             public float undersideClearance;
19             public float rearTaperRatio;
20
21             public float[] ToArray()
22             {
23                 return new[]
24                 {
25                     aspectRatio, slendernessRatio, frontalBlockageRatio, volumeEfficiency,
26                     surfaceRoughness, symmetryScore, undersideClearance, rearTaperRatio
27                 };
28             }
29         }
30
31         public struct AnalysisResult
32         {
33             public bool valid;
34             public float overallScore;
35             public string grade;
36             public GeometricFeatures features;
37             public string featureBreakdown;
38             public string improvements;
39             public float predictedCdLow;
40             public float predictedCdHigh;
41             public float separationRisk;
42             public float downforcePotential;
43             public float efficiencyScore;
44         }
45
46         private AnalysisResult cachedResult;
47         private int lastAnalyzedModelId;
48
49         private static readonly List<MeshFilter> MeshCache = new List<MeshFilter>(128);
50         private static readonly List<SkinnedMeshRenderer> SkinnedCache = new List<
```

```

SkinnedMeshRenderer>(32);

51
52     public bool TryGetResult(out AnalysisResult result)
53     {
54         result = cachedResult;
55         return cachedResult.valid;
56     }
57
58     public void AnalyzeModel(GameObject model, Bounds tunnelBounds, Vector3 flowDirection)
59     {
60         if (model == null) return;
61
62         int modelId = model.GetInstanceID();
63         if (modelId == lastAnalyzedModelId && cachedResult.valid) return;
64         cachedResult = default;
65         lastAnalyzedModelId = modelId;
66
67         var renderers = model.GetComponentsInChildren<Renderer>(true);
68         if (renderers == null || renderers.Length == 0) return;
69
70         Bounds modelBounds = renderers[0].bounds;
71         for (int i = 1; i < renderers.Length; i++)
72             modelBounds.Encapsulate(renderers[i].bounds);
73
74         if (modelBounds.size.sqrMagnitude < 1e-6f) return;
75
76         if (flowDirection.sqrMagnitude < 1e-6f) flowDirection = Vector3.right;
77         flowDirection.Normalize();
78         Vector3 upAxis = Vector3.up;
79         Vector3 sideAxis = Vector3.Cross(upAxis, flowDirection).normalized;
80         if (sideAxis.sqrMagnitude < 1e-6f) sideAxis = Vector3.right;
81
82         GeometricFeatures features = ExtractFeatures(model, modelBounds, tunnelBounds,
83         flowDirection, upAxis, sideAxis);
84         float[] featureArray = features.ToArray();
85
86         AeroMLPredictor.PredictionResult mlResult = AeroMLPredictor.Predict(featureArray);
87         float[] importance = AeroMLPredictor.ComputeFeatureImportance(featureArray);
88
89         float overallScore = ComputeOverallScore(features, mlResult);
90         string grade = ScoreToGrade(overallScore);
91         string breakdown = BuildFeatureBreakdown(features, importance);
92         string improvements = BuildImprovements(features, mlResult, importance);
93
94         cachedResult = new AnalysisResult
95         {
96             valid = true,
97             overallScore = overallScore,
98             grade = grade,
99             features = features,
100             featureBreakdown = breakdown,
101             improvements = improvements,
102             predictedCdLow = mlResult.predictedCd * 0.85f,
103             predictedCdHigh = mlResult.predictedCd * 1.15f,
104             separationRisk = mlResult.separationRisk,
105             downforcePotential = mlResult.downforcePotential,
106             efficiencyScore = mlResult.efficiencyScore
107         };
108
109     public void InvalidateCache()
110     {
111         cachedResult = default;
112         lastAnalyzedModelId = 0;
113     }
114
115     private GeometricFeatures ExtractFeatures(
116         GameObject model, Bounds modelBounds, Bounds tunnelBounds,
117         Vector3 flowDir, Vector3 upAxis, Vector3 sideAxis)
118     {
119         float length = ProjectSize(modelBounds, flowDir);
120         float height = ProjectSize(modelBounds, upAxis);

```

```

121     float width = ProjectSize(modelBounds, sideAxis);
122     float maxCross = Mathf.Max(height, width);
123
124     float aspectRatio = height > 1e-4f ? length / height : 1f;
125     float slenderness = maxCross > 1e-4f ? length / maxCross : 1f;
126
127     float frontalArea = height * width;
128     float tunnelCross = ProjectSize(tunnelBounds, upAxis) * ProjectSize(tunnelBounds,
sideAxis);
129     float blockage = tunnelCross > 1e-4f ? frontalArea / tunnelCross : 0f;
130
131     float volumeEff = ComputeVolumeEfficiency(model, modelBounds);
132     float roughness = ComputeSurfaceRoughness(model);
133     float symmetry = ComputeSymmetryScore(model, modelBounds, sideAxis);
134     float clearance = ComputeUndersideClearance(model, modelBounds, tunnelBounds, upAxis);
135     float rearTaper = ComputeRearTaperRatio(model, modelBounds, flowDir);
136
137     return new GeometricFeatures
138     {
139         aspectRatio = aspectRatio,
140         slendernessRatio = slenderness,
141         frontalBlockageRatio = Mathf.Clamp01(blockage),
142         volumeEfficiency = Mathf.Clamp01(volumeEff),
143         surfaceRoughness = Mathf.Clamp01(roughness),
144         symmetryScore = Mathf.Clamp01(symmetry),
145         undersideClearance = Mathf.Clamp01(clearance),
146         rearTaperRatio = Mathf.Clamp01(rearTaper)
147     };
148 }
149
150 private static float ProjectSize(Bounds bounds, Vector3 axis)
151 {
152     Vector3 absAxis = new Vector3(Mathf.Abs(axis.x), Mathf.Abs(axis.y), Mathf.Abs(axis.z));
153     return 2f * Vector3.Dot(bounds.extents, absAxis);
154 }
155
156 private float ComputeVolumeEfficiency(GameObject model, Bounds bounds)
157 {
158     float bbVolume = bounds.size.x * bounds.size.y * bounds.size.z;
159     if (bbVolume < 1e-6f) return 0.5f;
160
161     float meshVolume = 0f;
162     Transform root = model.transform;
163     MeshCache.Clear();
164     root.GetComponentsInChildren(true, MeshCache);
165
166     int sampleLimit = 6000;
167     int totalTris = 0;
168
169     for (int m = 0; m < MeshCache.Count; m++)
170     {
171         MeshFilter mf = MeshCache[m];
172         if (mf == null || mf.sharedMesh == null) continue;
173         Mesh mesh = mf.sharedMesh;
174         Matrix4x4 ltw = mf.transform.localToWorldMatrix;
175         Vector3[] verts = mesh.vertices;
176         if (verts == null || verts.Length < 3) continue;
177
178         for (int sub = 0; sub < mesh.subMeshCount; sub++)
179         {
180             if (mesh.GetTopology(sub) != MeshTopology.Triangles) continue;
181             int[] tris = mesh.GetIndices(sub);
182             int stride = Mathf.Max(1, tris.Length / 3 / Mathf.Max(1, sampleLimit - totalTris
));
183
184             for (int t = 0; t < tris.Length; t += 3 * stride)
185             {
186                 if (tris[t] >= verts.Length || tris[t + 1] >= verts.Length || tris[t + 2] >=
verts.Length) continue;
187                 Vector3 a = ltw.MultiplyPoint3x4(verts[tris[t]]);
188                 Vector3 b = ltw.MultiplyPoint3x4(verts[tris[t + 1]]);
189                 Vector3 c = ltw.MultiplyPoint3x4(verts[tris[t + 2]]);

```

```
190         meshVolume += SignedTetraVolume(a, b, c) * stride;
191         totalTris++;
192     }
193 }
194 }
195
196 meshVolume = Mathf.Abs(meshVolume);
197 return Mathf.Clamp01(meshVolume / bbVolume);
198 }
199
200 private static float SignedTetraVolume(Vector3 a, Vector3 b, Vector3 c)
201 {
202     return Vector3.Dot(a, Vector3.Cross(b, c)) / 6f;
203 }
204
205 private float ComputeSurfaceRoughness(GameObject model)
206 {
207     Transform root = model.transform;
208     MeshCache.Clear();
209     root.GetComponentsInChildren(true, MeshCache);
```

Listing E.1: Excerpt from the geometry analyzer

Appendix F

Project Save Manager Excerpt

This short appendix documents the project persistence layer used to save and recover analysis state.

```
1 using System;
2 using System.IO;
3 using System.IO.Compression;
4 using System.Collections;
5 using UnityEngine;
6
7 namespace FluidWorks.ProjectSystem
8 {
9     public class ProjectSaveManager : MonoBehaviour
10     {
11         // Add references to actual solver/view managers here in the real engine
12
13         /// <summary>
14         /// Initiates the saving process to package the .fluidworks file
15         /// </summary>
16         public void SaveProject(string targetArchiveFile, ProjectData currentData, string
sourceModelFilePath)
17         {
18             StartCoroutine(SaveRoutine(targetArchiveFile, currentData, sourceModelFilePath));
19         }
20
21         private IEnumerator SaveRoutine(string targetArchiveFile, ProjectData projectData, string
sourceModelFilePath)
22         {
23             Debug.Log($"[SaveManager] Starting save to {targetArchiveFile}...");
24             string tempDirName = "TempFluidWorksProject_" + Guid.NewGuid().ToString();
25             string tempDir = Path.Combine(Application.temporaryCachePath, tempDirName);
26
27             if (Directory.Exists(tempDir))
28                 Directory.Delete(tempDir, true);
29             Directory.CreateDirectory(tempDir);
30
31             // 1. Capture Preview
32             string previewPath = Path.Combine(tempDir, "preview.png");
33             bool captureComplete = false;
34             yield return StartCoroutine(PreviewCapture.CaptureScreenshotCoroutine(previewPath, () =>
captureComplete = true));
35
36             // Wait firmly just in case
37             yield return new WaitUntil(() => captureComplete);
38
39             try
40             {
41                 // 2. Copy the active 3D Model into the container
42                 if (!string.IsNullOrEmpty(sourceModelFilePath) && File.Exists(sourceModelFilePath))
43                 {
44                     string modelFileName = Path.GetFileName(sourceModelFilePath);
45                     string destModelPath = Path.Combine(tempDir, modelFileName);
46                     File.Copy(sourceModelFilePath, destModelPath, true);
47                 }
48             }
49         }
50     }
51 }
```



```
48         // The project data JSON should reference the relative file inside the ZIP
49         projectData.geometry.modelPath = modelFileName;
50     }
51
52     // 3. Serialize JSON Data
53     string jsonContent = JsonUtility.ToJson(projectData, true);
54     string jsonPath = Path.Combine(tempDir, "project.json");
55     File.WriteAllText(jsonPath, jsonContent);
56
57     // Create optional directories as per requirements
58     Directory.CreateDirectory(Path.Combine(tempDir, "screenshots"));
59
60     // 3.5 Include Generated Report if it exists
61     string reportPdfSource = Path.Combine(Application.persistentDataPath, "
GeneratedReport.pdf");
62     if (File.Exists(reportPdfSource))
63     {
64         File.Copy(reportPdfSource, Path.Combine(tempDir, "GeneratedReport.pdf"), true);
65         Debug.Log("[SaveManager] Included GeneratedReport.pdf in project archive.");
66     }
67
68     // 4. Zip the temp directory
69     if (File.Exists(targetArchiveFile))
70     {
71         File.Delete(targetArchiveFile);
72     }
73
74     string targetDir = Path.GetDirectoryName(targetArchiveFile);
75     if (!string.IsNullOrEmpty(targetDir) && !Directory.Exists(targetDir))
76     {
77         Directory.CreateDirectory(targetDir);
78     }
79
80     // Package everything into .fluidworks
81     ZipFile.CreateFromDirectory(tempDir, targetArchiveFile);
82     Debug.Log($"[SaveManager] Project packaged cleanly to {targetArchiveFile}");
83 }
84 catch (Exception ex)
85 {
86     Debug.LogError($"[SaveManager] Failed to save project: {ex.Message}");
87 }
88 finally
89 {
```

Listing F.1: Project save manager implementation

Appendix G

Project Data Model Excerpt

The data model excerpt shows how project metadata and saved simulation configuration are structured inside the application.

```
1 using System;
2 using UnityEngine;
3
4 namespace FluidWorks.ProjectSystem
5 {
6     [Serializable]
7     public class ProjectMetadata
8     {
9         public string projectName;
10        public string creationDate;
11        public string softwareVersion;
12    }
13
14    [Serializable]
15    public class GeometryData
16    {
17        public string modelPath;
18        public Vector3 modelScale;
19        public Vector3 modelRotation;
20        public Vector3 boundingBoxSize;
21        public int triangleCount;
22    }
23
24    [Serializable]
25    public class SolverParameters
26    {
27        public string solverType;
28        public int gridSizeX;
29        public int gridSizeY;
30        public int gridSizeZ;
31        public float viscosity;
32        public float density;
33        public float timeStep;
34        public int iterationCount;
35    }
36
37    [Serializable]
38    public class SimulationDiagnostics
39    {
40        public float meanVelocity;
41        public float maxVelocity;
42        public float pressureDrop;
43        public float wallShear;
44        public float vorticity;
45        public float divergenceError;
46    }
47
48    [Serializable]
49    public class VisualizationState
50    {
```

```
51     public float slicePlanePosition;
52     public float streamlineDensity;
53     public int selectedColormap;
54     public Vector3 cameraPosition;
55     public Quaternion cameraRotation;
56     public float cameraZoom;
57 }
58
59 /// <summary>
60 /// Master serializable container for all FluidWorks project state
61 /// </summary>
62 [Serializable]
63 public class ProjectData
64 {
65     public ProjectMetadata metadata;
66     public GeometryData geometry;
67     public SolverParameters solver;
68     public SimulationDiagnostics diagnostics;
69     public VisualizationState visualization;
70
71     public ProjectData()
72     {
73         metadata = new ProjectMetadata();
```

Listing G.1: Project data model implementation